

Running the server as a container

The published image is `registry.badassium.com/fatalexception/yarg-song-server`, tagged `latest` and the commit short sha. It is a distroless static binary — **8.6 MB**, no shell, running as `nonroot`.

This document records the first real deployment, on vault2, 2026-09-05.

What this deployment proved, that nothing had proved before

The multi-arch CI job verifies the image config's architecture and the ELF machine type of the binary inside. Both are checks on *bytes*. Until this deployment **the published image had never actually been run** — not by CI, not by anyone. It now has:

```
library indexed songs=22 distinct_charts=22 duplicate_packages=0 problems=0 took=7ms
listening addr=:8080 songs=/songs data=/data version=827e0fe
```

`version=827e0fe` is how you know the image is the commit it claims to be.

It also moved the server off the development machine for the first time. Every earlier measurement was `127.0.0.1`, which cannot fail in any of the ways a real network can.

The deployment

```
# on the host
mkdir -p /mnt/cache/appdata/yarg-song-server/songs \
         /mnt/cache/appdata/yarg-song-server/data
chown -R 65532:65532 /mnt/cache/appdata/yarg-song-server/data

docker run -d --name yarg-song-server \
  --restart unless-stopped \
  -p 8099:8080 \
  -v /mnt/cache/appdata/yarg-song-server/songs:/songs:ro \
  -v /mnt/cache/appdata/yarg-song-server/data:/data \
  registry.badassium.com/fatalexception/yarg-song-server:latest
```

> `:latest` **CHANGED MEANING ON 2026-09-08, and everything written below this line predates that.** It used to follow the default branch, so `:latest` was "the newest commit on main" and the deployment records further down confirm it that way — correctly, for the day they were written. It now means **the newest release tag**. The tags are:
>
> | Tag | Means |
> |---|---|
> | `:<short sha>` | that exact build, eight characters, never moves — **this is what a deployment pins** |
> | `:main` | the default branch head; the honest replacement for what `:latest` used to be |
> | `:vX.Y.Z` | a release, built from that tag's own commit |
> | `:latest` | the newest **release**, which is deliberately older than main most of the time |
>
> The `docker run` above uses `:latest` only to get a first container onto the box. **Pin the eight-character sha for anything you intend to keep**, exactly as the rest of this document does.
>
> **A digest identifies a BUILD, not a commit.** Commit `6c686ce` was built twice on 2026-09-08 — once from `ci/tag-images`, once from `main` — with an identical version string (`v0.1.0-5-g6c686ce`) and identical source, and the two images have different digests (`e982e528…` and `ee21f46f…`). Container images are not bit-reproducible here the way the release archives are: the image config carries build timestamps and nothing sets `SOURCE_DATE_EPOCH`. So the digest comparison used further down proves "this tag and that tag are the same image", which is what it was used for — it does **not** prove "this image is the only one that commit can produce". The `:<sha>` tag resolves to whichever build of that commit pushed last.
>
> That is not hypothetical — it is true in the registry right now. Commit `1ef7598e` was built by its `main` pipeline at 16:56 and again by the `v0.1.1` tag pipeline at 17:06, so today:
>
> `:1ef7598e a127c732… the tag build, pushed second :v0.1.1 a127c732… same image :latest a127c732… same image :main 9b1d0f03… the earlier build of THE SAME COMMIT`
>
> `:main` and `:1ef7598e` disagree while naming one commit. Neither is wrong.

Three things there are not decoration:

- **`chown 65532`.** The image is distroless and runs as `nonroot`, uid 65532. `/data` holds the on-demand pack cache and must be writable by that uid or the server starts fine and then fails the first time somebody asks for a song that is stored as anything but a `.sng` — a loose folder, a `.zip` or a `.7z`, all of which are packed on demand. `/songs` stays root-owned and is mounted `:ro`, because the server never writes to the library and should not be able to.
- **`/mnt/cache/...`, not `/mnt/user/...`.** An absolute pool path, not the shfs union. This host has a documented history of SQLite-on-FUSE corruption, and `/mnt/user` was **99% full** at deployment (568 GB of 29 TB) while the cache pool had 1.4 TB free.
- **`-p 8099:8080`.** 8099 was free; the container listens on 8080 internally. Checked against `ss -ltn` rather than assumed — this host had 59 containers running and 76 ports already in LISTEN.

Registry authentication

The host's stored credential was stale and `docker pull` failed with `unauthorized: HTTP Basic: Access denied`. A GitLab personal access token with the `api` scope works for a registry pull — measured; `read_registry` is the documented scope but was not required here.

Log in without putting the secret in `argv`, where `ps` would show it:

```
docker login registry.badassium.com -u <user> --password-stdin <<'ENDTOK'
<token>
ENDTOK
```

The token comes from 1Password at run time and is never written to a file, a commit, or this document.

End-to-end result, 2026-09-05

Step	Result
Pull and start on vault2	running, 0 restarts, indexed 22 songs in 7 ms
`GET /healthz` from the host	200
`yarg-sync` from ENG-1 over Tailscale	22 downloaded, 0 failed, 229,515 bytes in **341 ms**
Byte count vs the localhost run	**identical** — same archives, different machine
Second sync	0 downloaded, 22 already present, **0 bytes**, 19 ms
Unmodified YARG on the synced folder	**20 accepted, 2 refused**

The two refusals were the same two, for the same reasons, as every earlier run — `no notes found` and `no audio accompanying the chart file`, both independently flagged by our own scanner.

The client reached the server by its **Tailscale name**, not a LAN literal, so the same command works from anywhere ENG-1 happens to be. A `192.168.2.x` address would have worked at the time of the test and failed from the office.

The pack cache bound, verified here rather than in a test

`pack_cache_max` shipped with six red-proofed tests and did not work. Every test inserted, so every test reached eviction through the insert path; this deployment's cache was already full, every request was a hit, nothing was ever packed, and the bound was never enforced. Deploying it found that in under a minute.

After the fix (`6bcf4ec`, enforce at start as well as on insert), running deliberately tight at **102,400 bytes** against a library that packs to 229,515:

Moment	Cache
Before restart, from the broken build	229,515 bytes, 22 archives

Moment	Cache
Immediately after start, having served nothing	**67,681 bytes, 6 archives**
After serving all 22 songs	67,681 bytes, 6 archives
Songs the client received	**22 of 22, 0 failed**

Three things in that table, in order of importance:

- The cache dropped from 22 archives to 6 **before a single request arrived**. That is the start-up enforcement, and it is the case the tests all missed.
- It stayed at 6 while serving 22 songs, so eviction kept pace with insertion rather than merely happening once.
- **Every song was still delivered and verified.** Sixteen of the twenty-two had to be re-packed mid-sync because eviction had removed them, and the client — which re-derives each song's identity from the bytes it receives — accepted all of them. That is the concrete evidence that eviction costs a re-pack and never data.

> **The conclusion drawn from that third bullet was too strong, and the number in it was a clue nobody read.** "Eviction costs a re-pack and never data" is true; "and therefore the re-packed archive is the same archive" does not follow, and was false. Those same sixteen songs came back with *different bytes* — the packer drew a random mask per call — which the client could not notice, because it verifies the chart's identity and the chart is copied byte for byte. Identity survived; the archive did not. The next day the same sixteen turned up as sixteen SHA-256 mismatches between two machines, and that is what named the defect. Fixed on 2026-09-06 by deriving the mask; `docs/TEST-CORPUS.md` has the run. **A client that verifies identity will accept a byte stream that a cache, an `ETag` and a `Range` resume all require to be stable — so client acceptance is not evidence of stability.**

What this still does not prove

- ~~**arm64 has never been executed.**~~ **Closed 2026-09-06.** Both the cross-compiled arm64 binary and the published arm64 container image ran on a Raspberry Pi 4 Model B Rev 1.5 (`aarch64` , Debian 13 trixie), each indexing the 22-song corpus in about 25 ms, and the archives they served were byte-for-byte identical to every x86-64 sync. See `docs/TEST-CORPUS.md` , "The ARM leg". Note the caveats there: it was a Pi 4 rather than the 3B+ (that board is dead), the machine was one-shot and wiped, and the image was transported by `docker save` / `load` rather than pulled directly on the Pi.
- ~~**The Phase 2b exit criterion needs two clients.**~~ **Closed 2026-09-06.** YARG was installed on r7-desktop and both machines were scanned: 20 accepted, 2 refused on each, the same two songs. Doing it is what found the determinism defect above — see `docs/TEST-CORPUS.md` , fourth oracle run.
- ~~**One library, 22 songs, 224 KB.**~~ **Partly closed 2026-09-06.** Index time is linear at 0.57-0.59 ms/song and memory fits `13 MB + 6.5 KB/song` , measured at 1,000 / 10,000 / 31,109 songs; packing streams, with resident memory flat at 16.5 MB while serving 1.17 GB, and the cache bound held at 11x oversubscription. See `docs/SCALE.md` . **Still open:** every song in that experiment is synthetic, so a real library's *variety* is untested, and the oracle has never run at scale - which is the measurement most likely to find a rejection category we do not flag.

Re-deployed 2026-09-06 on `c623dae`, and what it proved beyond the fix

Every number above was taken from a server that packed with a random mask (`6bcf4ec` was running; an earlier revision of this document said `827e0fe`, which was the deployment before it). Pipeline 2259 published `c623dae` and the host was re-pulled:

```
docker pull registry.badassium.com/fatalexception/yarg-song-server:latest
docker rm -f yarg-song-server
rm -rf /mnt/cache/appdata/yarg-song-server/data/packs   # start with an empty cache on purpose
docker run -d ...                                       # identical arguments to the first deploy
```

The stored registry credential from 2026-09-05 was still valid, so no fresh login was needed. Wiping the pack cache first is deliberate: every archive is then packed by the new binary, and nothing served afterwards is a leftover of the random-mask era.

```
library indexed songs=22 distinct_charts=22 duplicate_packages=0 problems=0 took=11ms
pack cache bounded max_bytes=2147483648
listening addr=:8080 songs=/songs data=/data version=c623dae
```

Then five independent syncs were compared file by file:

Client	Server	Result
ENG-1	ENG-1, built from the working tree (windows/amd64)	reference
ENG-1	same, after wiping the whole pack cache	identical
r7-desktop	ENG-1's server	identical
ENG-1	**the vault2 container (linux/amd64)**	**identical**
r7-desktop	**the vault2 container (linux/amd64)**	**identical**

110 archives, two client machines, two servers built for different operating systems and by different toolchains, and one deliberate cache wipe — all one set of bytes. **The derivation is not platform-dependent**, which the fix needed to be but which nothing had shown until this run. Unmodified YARG on the container's output: **20 accepted, 2 refused**, the same two songs.

Worth stating plainly because it is the standard this project holds: the last row of that table is an identity established by hashing every file, not by arguing that identical inputs must give identical results.

Re-deployed 2026-09-06 on `423902b`, with archive ingest

The library on the host was replaced with the 23-case corpus, which includes `23-zipped.zip` — a song delivered as an archive rather than a folder — and the pack cache was wiped so nothing served afterwards was a leftover of the previous image.

```
library indexed songs=23 distinct_charts=23 duplicate_packages=0 problems=0 took=9ms
listening addr=:8080 songs=/songs data=/data version=423902b
```

Check	Result
`/version`	`423902b`
`/healthz`	200
The archive-sourced song in the catalog	`name="Zipped"`, `source_path="23-zipped.zip"`, **0 issues**
`yarg-sync` from an empty folder	23 downloaded, 0 failed, 238,215 bytes in 399 ms
Every archive vs the locally built server	**23 / 23 identical**

That last row is the container-independence property crossing a build platform: the song the **linux/amd64 container** ingested from inside a `.zip` is byte-identical to the one a **windows/amd64** build produced from the loose folder. Nothing about the container it arrived in, or the machine that packed it, reaches the bytes a client receives.

The host's stored registry credential was still valid, so no fresh login was needed.

Re-deployed 2026-09-06 on `077f36e`, and the hostile-archive fix measured on the real thing

Pipeline 2278 published `077f36e` (archive-ingest hardening). Two things were worth measuring here that a test cannot reach: whether a code change altered the bytes a client receives, and whether the new "this archive is unreadable" report actually arrives where an operator would see it.

`latest` **was confirmed to be the commit before it was run**, not assumed:

```
docker pull …:latest
docker pull …:077f36eb        # note EIGHT characters - CI tags with CI_COMMIT_SHORT_SHA
docker image inspect …:077f36eb --format '{{.Id}}'   # same image id as :latest
```

That the tag is eight hex characters and the git short sha is seven is a small trap: a `:077f36e` pull fails with `manifest unknown`, which reads like a missing image rather than a mistyped tag. `GET /api/v4/projects/53/registry/repositories/14/tags` lists what exists.

Nothing about a code change reached the bytes

The pack cache was hashed **before** the upgrade, wiped, and hashed again after the new binary had re-packed all 23 songs:

```
sha256sum *.sng | sort -k2 > /tmp/packs-423902b.txt   # before
# … pull, rm -f, rm -rf data/packs, docker run …
sha256sum *.sng | sort -k2 > /tmp/packs-077f36e.txt   # after
diff /tmp/packs-423902b.txt /tmp/packs-077f36e.txt    # no output
```

Check	Result
`/version`	`077f36e`
`/healthz`	200
Index	23 songs, 23 distinct charts, **0 problems**, 9 ms
23 archives re-packed on a wiped cache, vs the `423902b` cache	**byte-for-byte identical**
`yarg-sync` from ENG-1 over Tailscale	23 downloaded, 0 failed, 238,215 bytes in 312 ms
The 23 files the client received, hashed, vs the 23 packs on the server	**identical set**

The determinism property now holds **across a code change**, which is a different claim from the ones already recorded — those compared machines, operating systems and architectures at a fixed commit. Byte-stability across upgrades is what a client's `ETag` and `Range` resume actually depend on in practice, since a server gets upgraded far more often than it changes CPU.

The silently-ignored song, reproduced and then observed being reported

`077f36e` exists because a zip written with **backslash** separators used to disappear from a library with no message. A probe archive was built to be exactly that shape — `song.ini`, `notes.mid` and `song.ogg` under `Backslash Song\`, written with the separator stored verbatim — dropped into the host's library, and the server restarted.

Building the probe is itself instructive. Python's `zipfile` rewrites `os.sep` to `/` in `ZipInfo.__init__`, so `ZipInfo("Backslash Song\\song.ini")` silently produces a *forward* slash and the probe proves nothing; the name has to be assigned after construction. And `namelist()` normalises backslashes when reading back, so the file looks wrong even when it is right — the only honest check is to count `0x5C` bytes in the file itself. **Both the writer and the reader hide the thing being tested**, which is the same shape as the defect: a convenient view of an archive is not the archive.

```
level=INFO  msg="library indexed" songs=23 distinct_charts=23 duplicate_packages=0 problems=1
level=WARN  msg="could not index" path=24-backslash.zip
            err="scan: archive contains a song.ini or a chart but no song folder could be read
                 from it; if it was made by an old Windows tool it may use backslash path
                 separators - re-zipping it will fix that"
```

It appears in `/api/v1/library` under `problems` as well as in the log, so an operator with no shell on the host still sees it. Before this commit the same file produced `problems=0` and no mention anywhere.

The probe was removed afterwards and the library restored to the 23-case corpus — a permanent `problems=1` would make every future index report ambiguous. `0 problems`, `restarts=0`, health 200 at the end.

Re-deployed 2026-09-07 on `64b42af`, and the first test with real concurrent clients

`64b42af` is the build that stopped the server 404-ing songs that exist (see ADR-002, "Concurrency"). Deployed the same way: confirm the tag is the commit by image id, wipe the pack cache, recreate.

Take the eight-character tag from `git rev-parse`, do not type it from memory. The image tag is `CI_COMMIT_SHORT_SHA` — eight characters — and the git short sha is seven. Guessing the eighth produced `64b42af7` and a `manifest unknown`, which reads like a missing image rather than a typo. The sha is `64b42af1b368…`, so the tag is `64b42af1`, and its image id matched `:latest` exactly.

Check	Result
`/version`	`64b42af`
`/healthz`	200, `restarts=0`
Index	23 songs, 23 distinct charts, **0 problems**, 15 ms
23 archives re-packed on a wiped cache, vs the `077f36e` cache	**byte-for-byte identical**

That is determinism holding across a **second** code change, on top of the machines, operating systems and architectures already covered.

Four real clients at once, over the network

Every concurrency measurement until now used goroutines against an in-process test server. Four `yarg-sync` processes were run simultaneously from ENG-1 against the vault2 container over Tailscale:

Client	Result
1	23 downloaded, 0 failed, 238,215 bytes, 456 ms
2	23 downloaded, 0 failed, 238,215 bytes, 448 ms
3	23 downloaded, 0 failed, 238,215 bytes, 483 ms
4	23 downloaded, 0 failed, 238,215 bytes, 449 ms

Every file byte-identical across all four clients, compared by SHA-256 per filename rather than by count or total. Four simultaneous clients cost about 40 ms each against the ~410 ms a single client took earlier, so nothing here is serialising badly.

What this run does NOT test, stated because it would be easy to read it as more than it is: this deployment's cache bound is 2 GB against a 238 KB library, so **nothing is ever evicted** and the defect `64b42af` fixes could not fire here even on the old build. This exercises the concurrent *serve* path over a real network; the eviction race is covered by `internal/httpapi/concurrency_test.go`, which has to bound the cache to a single archive to provoke it at all. A Pi with a small SD card and a large library is where the two conditions meet, and that combination is still unmeasured on real hardware.

Upgrade to `a5f98c4`, 2026-09-07 — the pack-cache race and the chart checks

Pipeline 2295. Two changes worth deploying: the eviction race in `packcache` that answered **500 for a song that exists** on Linux (see `docs/ADR-002-v1-store.md`), and the two new chart issues, `chart_unreadable` and `chart_truncated`.

`:latest` was confirmed to BE the commit before anything was recreated, by image id rather than by trust:

```
docker pull -q …:latest
docker pull -q …:a5f98c41     # EIGHT characters; CI tags with CI_COMMIT_SHORT_SHA
docker image inspect …:latest    --format '{{.Id}}'   # sha256:b2bc48bb…
docker image inspect …:a5f98c41  --format '{{.Id}}'   # sha256:b2bc48bb…  same
```

The container was pinned to `:a5f98c41` rather than `:latest`, so `/version` and the running image agree even after the next pipeline publishes.

Determinism held across a fourth code change

The pack cache was hashed before the upgrade, wiped, and every song re-fetched so the new binary re-packed all 23:

```
archives before: 23  after: 23
BYTES IDENTICAL across the code change
```

That is the fourth code change this has been checked across, and the check is cheap enough that there is no reason to stop doing it.

Check	Result
`/version`	`a5f98c4`
`/api/v1/library`	23 songs, 23 distinct charts, `problems: null`
Browse page `GET /`	200, 9,365 bytes
`GET /nope`	404 — the catch-all trap has not come back
Pack cache after re-fetch	23 archives, byte-identical to before

What this does NOT verify, and it is the important one: the defect this upgrade exists to fix cannot fire on this deployment. The bound here is 2 GB against a 225 KB library, so nothing is ever evicted, and the rename-then-open window only opens when eviction runs concurrently with packing. The fix is covered by `TestEvictionNeverStealsAnArchiveBeforeItsPackerCanOpenIt`, which has to bound the cache to a single archive to provoke it at all. **A green deployment here is not evidence about the race.** The place the two conditions actually meet is a Pi with a small card and a large library, and that is still unmeasured on real hardware.

Second deployment, 2026-09-08 — `a5f98c4` to `3f80da0`

The live server had drifted a day behind `main`, so it was missing the browse page's scan-health view among other things. Updated in place, verified rather than assumed.

The image tag is the EIGHT-character short sha, which is the thing most likely to trip somebody up: `3f80da0ad757…` publishes as `:3f80da0a`, not `:3f80da0`. A seven-character pull fails with `manifest unknown`, which reads like a missing image rather than a typo.

The old container's configuration was read, not recalled. `docker inspect` gave the exact `Cmd`, port bindings, restart policy and mounts, and the new container was created from those rather than from the example earlier in this document. An example in a doc is a record of what was run once, not a description of what is running now.

```
docker pull registry.badassium.com/fatalexception/yarg-song-server:3f80da0a
docker rename yarg-song-server yarg-song-server-prev && docker stop yarg-song-server-prev
docker run -d --name yarg-song-server --restart unless-stopped -p 8099:8080 \
  -v /mnt/cache/appdata/yarg-song-server/songs:/songs:ro \
  -v /mnt/cache/appdata/yarg-song-server/data:/data \
  registry.badassium.com/fatalexception/yarg-song-server:3f80da0a \
  --songs /songs --data /data --listen :8080
```

The previous container is renamed and stopped, not deleted. Rolling back is `docker rm yarg-song-server && docker rename yarg-song-server-prev yarg-song-server && docker start yarg-song-server`. Delete it once the new one has been trusted for a while.

Verified after the swap

Check	Result
`/version`	`3f80da0` — the image is the commit it claims to be
Startup log	`songs=23 distinct_charts=23 duplicate_packages=0 problems=0 took=19ms`
Browse page carries the health machinery	all four markers present
Browse page still self-contained	**0** external references
A real song downloads	`200`, 8,471 bytes, begins `SNGPKG`
`/nope`	`404` — the root is still an exact match, not a catch-all

The last two matter more than they look. A deployment that serves a page but not a song is a deployment that passes a health check and fails a player; and the `GET /{$}` versus `GET /` distinction is the kind of thing an image rebuild could quietly undo, turning every 404 into HTML that a sync client would try to parse as a `.sng`.

State check over Tailscale, 2026-09-08 13:52Z — read-only, nothing deployed

Reached vault2 as `vault2.tail20252e.ts.net` rather than `192.168.2.7`; the Tailscale name works off-LAN and the LAN literal does not.

Measured	Value
`docker inspect`	`running`, started `2026-09-08T02:39:26Z`, **restarts=0**
Image	`registry.badassium.com/fatalexception/yarg-song-server:3f80da0a`
Ports	`8080/tcp -> 0.0.0.0:8099`
`GET /healthz` on `:8099`	`200`
Startup log	`songs=23 distinct_charts=23 duplicate_packages=0 problems=0 took=19ms`

Measured	Value
Commits behind `origin/main` (`3f80da0..7e12e55`)	**17**

The live deployment is healthy and has not restarted; it is simply old on purpose. Nothing here was changed — deploying, and turning `check_uploads` on, are both still Jay's calls.

The image is distroless, so `docker exec <name> sh -c ...` fails with `exec: "sh": executable file not found in $PATH`. That is the image behaving correctly, not a broken container. An authenticated `/api/v1/*` probe therefore cannot be run from inside the container; use `/healthz` (unauthenticated) plus the startup log, or call the API from a host that holds the key.

The registry's on-disk tag directory is NOT a reliable inventory

Listing `/var/opt/gitlab/gitlab-rails/shared/registry/docker/registry/v2/repositories/<ns>/<img>/_manifests/tags` inside the GitLab-CE container returned **69** tags and omitted `7e12e557` — the tag for `main`'s head, whose pipeline had succeeded 90 minutes earlier. The GitLab API returned **70**, including it. Read from the disk, the conclusion would have been "CI never published an image for HEAD", which is false and would have been the ninth wrong-instrument finding in this project.

Use the API, and take the digest, not the tag name:

```
GET /api/v4/projects/<id>/registry/repositories
GET /api/v4/projects/<id>/registry/repositories/<repo_id>/tags/<tag>   -> .digest
```

`latest` and `7e12e557` resolve to the same digest (`sha256:2597abd874b3b646…`), so `:latest` currently *is* head-of-`main` — but that is a measurement with a timestamp on it, not a property. Compare digests every time; a tag name is a label somebody can move.

Pipeline history for the 17 undeployed commits is 16 `success` and one `failed` (`e8eda83b`, a docs commit). Project id is **53**; the credential is Windows Credential Manager `dev:gitlab-ce-pat`, read via `CredRead` so the token never reaches a command line or the transcript.

Third deployment, 2026-09-08 — `3f80da0` to `027d331`, and v0.1.0

31 commits, covering the feature registry, the config menu and its persistence, `-trimpath`, and the release packaging. **Feature defaults are unchanged**, so this is a code update rather than a posture change: `browse_ui` on, `check_uploads` off, `config_writes` off.

The pull failed first, and the reason is a landmine this document has warned about in general terms. vault2's cached registry credential in `/root/.docker/config.json` had one auth entry and was refused (`denied: access forbidden`). `/root` does not survive a reboot on Unraid, so a cached docker login is not durable infrastructure — it is something that works until it doesn't, silently, at the moment somebody needs to deploy.

The 1Password item named `GitLab Registry Pull (vault2 deploy token)` is NOT for this project. Its username is `fallout-pull` and it is scoped to fallout-research: `docker login` succeeds with it and the very next `docker pull` is denied, which reads like a registry fault rather than a scope one. The generic name is the trap. What worked is the documented route — a GitLab PAT with `api` scope, `--password-stdin` so the token never reaches `argv`.

The login was removed again immediately after the pull (`docker logout`). An `api`-scoped token is far more than a pull needs, and leaving one cached on the box is a standing over-grant. The container runs from the local image afterwards and needs no registry access. **The right fix is a project-scoped `read_registry` deploy token for `fatalexception/yarg-song-server`, stored in 1Password under a name that says which project it belongs to** — that is a credential change and belongs to Jay.

```
docker pull registry.badassium.com/fatalexception/yarg-song-server:027d331f
docker rm -f yarg-song-server-prev            # the 16h-old rollback, now superseded
docker rename yarg-song-server yarg-song-server-prev && docker stop yarg-song-server-prev
docker run -d --name yarg-song-server --restart unless-stopped -p 8099:8080 \
  -v /mnt/cache/appdata/yarg-song-server/songs:/songs:ro \
  -v /mnt/cache/appdata/yarg-song-server/data:/data \
  registry.badassium.com/fatalexception/yarg-song-server:027d331f \
  --songs /songs --data /data --listen :8080
```

The container's configuration was **read from `docker inspect`, not recalled from this document** — an example here is a record of what was run once, not a description of what is running now.

Verified after the swap

Check	Result
`/version`	`027d331` — the image is the commit it claims to be
Startup log	`songs=23 distinct_charts=23 duplicate_packages=0 problems=0 took=19ms`
`/healthz`	200
`GET /`	200, 19,956 bytes (the page grew with the settings panel)
`GET /nope`	**404** — the root is still an exact match, not a catch-all
`POST /api/v1/check`	**404** — off, and still *absent* rather than forbidden
`PUT /api/v1/features/{name}`	**404** — writes off, so the route is not there
A real song	200, 8,471 bytes, begins `SNGPKG`

The last three matter most on this deployment. The routes for `check` and the feature writer are now **registered unconditionally** — that changed in this batch so features could be toggled at runtime — and the guarantee that a disabled feature is indistinguishable from an absent one is now kept by the handlers rather than by the router. Those two 404s are that guarantee, measured on the live server rather than only in a test.

`config_writes` is off here, so nobody can change this server's features over HTTP. Setting it to `local` would only help somebody with a shell on vault2 anyway; `lan` is the one that would matter, and on a box reachable from the whole house that is a decision rather than a convenience.

v0.1.0

Tagged at `027d331`. The `release:tag` job produced all six archives plus `SHA256SUMS` with `artifacts_expire_at = NEVER`, which is the whole point of tagging.

A tagged pipeline did NOT build a container image, and "skipped" understates it: `container-image` ran on the default branch and on `ci/*`, a tag has no `CI_COMMIT_BRANCH`, so the job was **absent from the pipeline entirely** — not listed with a reason. Pipeline 2383 is green with four check jobs and `release:tag` and nothing else. The image for this commit exists as `:027d331f` from the `main` pipeline, so nothing was missing that day, but `:v0.1.0` is not in the registry and never will be.

Fixed on 2026-09-08 by `- if: '$CI_COMMIT_TAG'`. It cannot be applied retroactively: a tag pipeline builds from the tag's own commit, and `v0.1.0` points at `027d331f`, which predates the rule. Re-running it reproduces the same imageless pipeline. **`v0.1.1` is the first release with an image.**

Fourth deployment, 2026-09-08 — `027d331` to `v0.1.1`, and the first release-pinned container

The first deployment that pins a **release** rather than a commit, and the first where `/version` answers with a version number instead of a sha.

Pinned by DIGEST, not by tag, and that is a change of practice with a reason measured the same day: the `:<short sha>` tag is NOT immutable. Commit `1ef7598e` was built twice — once by its `main` pipeline, once by the `v0.1.1` tag pipeline — and the second push moved `:1ef7598e` to the newer image, so `:main` and `:1ef7598e` now name one commit and resolve to different digests. A digest is the only reference that cannot move under a running deployment.

```
docker pull registry.badassium.com/fatalexception/yarg-song-server@sha256:a127c732b8b58614…
docker stop yarg-song-server && docker rename yarg-song-server yarg-song-server-prev
docker run -d --name yarg-song-server --restart unless-stopped -p 8099:8080 \
  -v /mnt/cache/appdata/yarg-song-server/songs:/songs:ro \
  -v /mnt/cache/appdata/yarg-song-server/data:/data \
  registry.badassium.com/fatalexception/yarg-song-server@sha256:a127c732b8b58614… \
  --songs /songs --data /data --listen :8080
```

Check	Before (`027d331f`)	After (`v0.1.1`)
Image id	`c45f8b9c…`	`f323f3b8…`, `arch=amd64`
`/version`	`027d331`	**`v0.1.1`**
`/healthz`	200	200
Index	23 songs, 23 charts, 0 problems	23 songs, 23 charts, 0 problems, **9 ms**
`restarts`	0	0
Features	`browse_ui` on, `check_uploads` off, `config_writes` off	unchanged

One song was fetched from the running release, not just counted: `dc079fac…` returned 200 and 8,471 bytes hashing to `e9948d1e85f9317b22fd95af9e3230224d8dc050adb0cc7570e2d774738022eb` — **byte-identical to the same song served by a local Linux build earlier that day**. Determinism now holds across the shipped release image as well as across machines and architectures.

The previous container is kept stopped as `yarg-song-server-prev` on `:027d331f`; rollback is `docker stop yarg-song-server && docker start yarg-song-server-prev` after a rename.

Registry auth is still an open gap. The pull was denied until a login, and the credential used was the `api`-scope GitLab PAT again. It was fed over SFTP to a `600` file, consumed by `--password-stdin`, shredded, and `docker logout` ran immediately — `/root/.docker/config.json` is 16 bytes afterwards, so **no registry credential is stored on vault2**. The durable fix is still a project-scoped `read_registry` deploy token, which nobody has created.

Current state, measured 2026-09-15

Read directly off the box rather than inherited from an earlier section of this document, because the two had drifted — everything above was written while the container was still on `027d331f`, and this file went on saying so.

```
yarg-song-server   Up 6 days   restarts=0   restart=unless-stopped
image   registry.badassium.com/fatalexception/yarg-song-server@sha256:a127c732…
        (= :v0.1.1 = :latest, pinned by DIGEST, not by a floating tag)
started 2026-09-08T21:20:06Z
ports   8099 -> 8080          /healthz = 200
mounts  /mnt/cache/appdata/yarg-song-server/songs -> /songs   ro
        /mnt/cache/appdata/yarg-song-server/data  -> /data    rw  (65532:65532)
args    --songs /songs --data /data --listen :8080
image size   10,183,116 bytes (9.7 MiB)
library      23 songs, 568 KB — still the mkcorpus TEST CORPUS, not a real library
```

So the "deploy vault2 to v0.1.1" item was already done on 2026-09-08 and had been sitting open in two places since. There is also a stopped `yarg-song-server-prev` on the old `027d331f`, kept deliberately as a rollback.

`/songs` **holds the 23-case synthetic corpus.** Pointing this at a real library is a mount change and a restart, not a migration — the server re-indexes on start and never writes to that directory.

The Unraid template (2026-09-15)

The container was an ORPHAN in Unraid's Docker tab for ten days. It was created with the hand-rolled `docker run` at the top of this document, and a container with no template gets no edit form, no update button and no autostart toggle — and a GUI "recreate" would not have known about the mounts at all. Nothing was broken; it simply could not be managed from the UI, and the only record of how to reproduce it was this file.

`deploy/unraid/my-yarg-song-server.xml` fixes that. It describes the container **exactly as it already runs**, so installing it changed nothing about the running service — verified after the install: `healthz=200`, still `Up 6 days`, `restarts=0`.

Installed and checked:

`xmllint --noout` on the template	VALID
the validator itself, against deliberate garbage	rejects it (so VALID means something)
`<Name>` matches the running container	`yarg-song-server`
`<DateInstalled>` vs `State.StartedAt`	both `2026-09-08T21:20:06Z`
Config `Target=` / `Mode=` vs the container's mounts	`/songs` ro, `/data` rw — match
Port Config vs `HostPort`	8099 — match
file mode vs the neighbouring templates	`600 root` — match
`docker update --restart unless-stopped`	set
`/var/lib/docker/unraid-autostart`	`yarg-song-server` added (72 entries → 73)

Autostart is two independent mechanisms and both are now set. The docker restart policy is what actually survives a reboot; the autostart file is what makes the WebUI toggle agree and sets start order. Setting only one gives a container that comes back but shows as disabled, or a toggle that lies.

The template uses `:latest`, not the digest the container currently runs. Deliberate: a digest cannot be updated from the UI, and `:latest` already means *the newest release* rather than the newest commit, which is the right thing for a button a person presses. The running container keeps its digest pin until something recreates it. For a deployment that must not move at all, put the digest back in the template's `<Repository>` and accept that the update button becomes decorative.

Two things the UI will do that look like faults

- **The Console button fails**: `exec: "sh": executable file not found in $PATH`. Measured, not assumed. The image is distroless — there is no shell in it. Use the WebUI, `/healthz`, or the container log.
- `:latest` **will look stale** whenever `main` has moved and no tag has been cut. That is the tag doing its job. Cut a release; never repoint `:latest` at a branch.

For anything that is not Unraid

`docker-compose.yml` at the repo root, and `deploy/README.md` for both paths. Compose works on Unraid too and is still the wrong answer there — a compose stack is an orphan in the Docker tab, which is the whole problem the template solves.

The YARG share, and a real library (2026-09-15)

Until now the container served `/mnt/cache/appdata/yarg-song-server/songs` — the 23-case synthetic corpus, 568 KB of scanner fixtures. Song content was living in `appdata`, which is where a *service* keeps its state, not where somebody's music belongs. It now has its own share.

```
share   YARG            /mnt/user/YARG
        songs/          the library the server serves, mounted READ-ONLY
        playlists/      reserved
        art/            reserved
```

Settings (`/boot/config/shares/YARG.cfg` , version-controlled at `deploy/unraid/YARG-share.cfg`):

`shareUseCache`	`yes` — writes land on the NVMe, the mover pushes to the array hourly
`shareCachePool`	`cache`
`shareAllocator`	`highwater`
`shareExport` / `shareSecurity`	`e` / `private` , write list `fatalexception`

Array, not cache-pinned, and that is deliberate. The house rule is that new *services* go on `/mnt/cache/...` — that exists because of SQLite-on-FUSE corruption and a full array, and neither applies to read-only song files. Charts are hand-made work; parity protection is worth more here than shfs overhead on a mount the server never writes to. The array had 2.8 TB free when this was decided (the "99% full / 567 GB" figure in older notes is from 2026-09-06 and is stale).

Creating a share by hand does NOT export it over SMB

The part worth keeping, because every visible signal said it had worked:

- Writing `/boot/config/shares/YARG.cfg` and `mkdir` ing `/mnt/user/YARG` is enough for **emhttpd** — the share appeared in `/usr/local/emhttp/state/shares.ini` with every setting correct.
- It is **not** enough for **samba**. `/etc/samba/smb-shares.conf` is generated by emhttpd, not by `rc.samba` , and it stayed at 25 stanzas with no `[YARG]` . The share existed, was configured, and was invisible to every Windows machine.

The supported trigger is the same command the SMB security form posts (`/usr/local/emhttp/plugins/dynamix/SecuritySMB.page` , `changeShareSecurity`):

```
emcmd
"changeShareSecurity=Apply&shareName=YARG&shareExport=e&shareSecurity=private&shareWriteList=fatalexception&shareR
eadList=&shareVolsizelimit="
```

Post the values that are **already in the cfg** and it is a no-op to the file and a trigger for the samba config. Measured: 25 → 26 stanzas, `[YARG]` present with `write list = fatalexception` , no other share's stanza disturbed, and `smbclient -L localhost` lists it — which is the far-end proof, not the file.

Two smaller traps met on the way:

- **Unraid quotes share names in `shares.ini`** (`["YARG"]`), so a grep for `^.YARG.$` finds nothing and reads as "emhttpd has not seen it". It had.
- **emhttpd rewrites share cfgs with CRLF.** A `diff` against an LF copy reports every line changed. `file` is what settles it; the content was identical.

128 real songs, copied over the LAN

`C:\dev\_incoming\YARG\library` on ENG-1 → `\\192.168.2.7\YARG\songs`:

```
Dirs : 129 copied    Files : 644 copied    Bytes : 1.162 g    FAILED : 0
42 seconds, ~28 MB/s
```

Over SMB, not through the device bridge — 1.2 GB against a ~50 s transfer budget is not viable, and both machines can already see each other. Originals on ENG-1 are untouched; this was a copy.

`extracted/` is not a duplicate of `library/`. Both hold 644 files and 1,190 MB, which is why it first looked like the same songs twice. It is the same 128 songs arranged as **3 pack folders** (`Alternative Pack Vol.2`, `Grateful Dead - Phish`, `METAL PACK`) rather than flat. Equal file counts and equal byte totals are not equal content; comparing the folder *names* is what said so. The flat layout was the one copied.

Result

```
container recreated on /mnt/user/YARG/songs:/songs:ro   (data mount unchanged)
image still pinned @sha256:a127c732…  (= v0.1.1)
healthz = 200        version=v0.1.1
library: songs=128  distinct_charts=128  duplicate_packages=0  problems=null
```

0 problems across 128 real charts — worth noting as a data point rather than a claim: it means the scanner accepted all of them, not that YARG would.

The synthetic corpus is **left in place** at `/mnt/cache/appdata/yarg-song-server/songs`. It is still what the oracle and the sync tests use, and it is 568 KB.

To add songs now: drop them in `\\vault2\YARG\songs` and restart the container. The server re-indexes on start and never writes to that directory.

Fifth deployment, 2026-10-30 — `v0.1.1` to `v0.1.2`, and the `;` fix

The first change to the server's Go code since `v0.1.1` (one of twelve commits; the rest are docs and deployment files). A song whose metadata held a `;` in a value — `charter = TheSheepQueen; Whizzer` — was in the catalogue and answered `GET /song/<hash>.sng` with **HTTP 500**, so a client syncing the library got 127 of 128 songs. The writer rejected `;` as "the format's own restriction"; the reference encoder (SngCli v0.3.0) packs it and decodes it back unchanged.

Superseded: the *Current state, measured 2026-09-15* block above. What runs now:

```
image    registry.badassium.com/fatalexception/yarg-song-
server@sha256:2d5ff48a089f965c68297cee9e278ca479d54965915b2e13aaf30abd7b59ba4a
         (= :v0.1.2 = :latest, pinned by DIGEST)         image id 92e54925…, 10,183,116 bytes, linux/amd64
/version v0.1.2      /healthz 200      restarts=0      policy unless-stopped      user nonroot
mounts   /mnt/user/YARG/songs -> /songs  ro      /mnt/cache/appdata/yarg-song-server/data -> /data  rw
args     --songs /songs --data /data --listen :8080          port 8099 -> 8080
index    songs=128 distinct_charts=128 duplicate_packages=0 problems=0     took 8.6 s at start
```

The previous container is kept stopped as `yarg-song-server-v0.1.1` (the `v0.1.1` digest, `a127c732…`). Rollback is `docker rename` the two and `docker start` the old one, about ten seconds. `yarg-song-server-prev` is something else: a container on `027d331f` that exited seven weeks ago, and not a rollback for anything current.

What was done, and how

A release tag first, because this deployment pins a release: `v0.1.2` on `8f8370b`, pipeline 3593, all six jobs green. The registry's `:v0.1.2` and `:latest` resolve to one digest; `:main` is a different digest for the same commit, which is the tag/branch split `Fourth deployment` measured.

The registry login, because vault2 keeps none. The token was read once from Windows Credential Manager and written over SFTP to a mode-600 file on `/dev/shm` (RAM, not disk), then fed to `docker login --password-stdin`, the file removed, the image pulled, and `docker logout` run from an `EXIT` trap. Checked afterwards, not assumed:

token at the source vs on vault2	51 chars, same sha256 prefix both ends — and no BOM (first bytes were not `efbbbf`)
token files left in `/dev/shm`	0
`/root/.docker/config.json`	16 bytes — no credential stored
pulled digest	equals the one the registry reported

The swap rolled back by itself if it had to. `stop`, `rename` to `-v0.1.1`, `run`, then a 30 s health loop that required `/healthz` 200 *and* `/version` containing `v0.1.2`; failing that it would `rm -f` the new container, rename the old one back and start it. It was healthy 10 s after the new container started, 8.6 s of which was the index of 128 songs. The time the old container took to stop was not measured, so ten seconds is a floor for the downtime, not the figure.

Verified against a baseline taken BEFORE the deploy

Every song was `HEAD`ed on the live `v0.1.1` first and the status, `ETag` and `Content-Length` saved. After the swap:

Check	Result
the 127 songs that already worked	**127 of 127 identical** — status, ETag and length
Roots Bloody Roots (`f5178d27…`)	**500 → 200**; ETag `cd5f41c5…` equals the catalogue's `package_hash`; 2,774,330 bytes
those bytes vs a build of the same commit made on Windows	**SHA-256 identical** (`a8ddaa13…`)
SngCli decode of the served archive	0 errors; `charter = TheSheepQueen; Whizzer` intact

Check	Result
`SHA1(notes.mid)` in the served archive	equals the catalogue's `chart_hash` — identity holds

The ETag is the package hash, so "127 ETags unchanged" means the change cannot have reached any package whose metadata has no `;` in a value, which is what the one-line diff said it would not.

Three instrument errors, kept because each read like a result

- `package_hash` **is not the SHA-256 of the file.** It was tried as a download check and failed even for a song that served fine. It is the ETag; the file bytes hash to something else.
- `@($null).Count` **is 1 in PowerShell**, so `problems=1` appeared for a library with `problems: null`. The raw field and the server's own startup log (`problems=0`) both said otherwise.
- **PowerShell variable names are case-insensitive** (already in `WORKSPACE.md`): a loop variable `$s` overwrote the path variable `$S` , and the baseline file silently failed to write — twice.

Still open

- **The game has not loaded the `;` package.** Everything above is the server and the reference encoder. The in-game check is a sync from a client that is one song short, then `server:` in the song list showing 128.
- **The container may not be Unraid-managed.** Its `net.unraid.docker.managed` label is empty, so the *Docker* tab may not track it or offer an update. The likely cause is that the `docker run` that recreated it for the YARG share on 2026-09-15 did not link it to the template — an inference: only the empty label was measured, and what the Docker tab actually shows was not looked at.
- **There is still no durable registry credential.** This deployment used the `api` -scope token again, carefully, rather than the project-scoped `read_registry` deploy token that would make it unnecessary.

v0.1.3-yargsong, 2026-10-31

The deploy that made YARN setlists visible. Index went **128 -> 609 songs**, all 481 added by `yarn-install` and indexed through the new `.yargsong` path.

Built on vault2, not pushed through the registry. Docker Desktop was not running on ENG-1, and the alternative - copying a binary into the running container - leaves a container whose image label is a lie. `git archive HEAD` to a tarball, extract on vault2, `docker build` with the repo's own Dockerfile. The build stage cross-compiles anyway, so nothing about this needed buildx or a second architecture.

```
docker run -d --name yarg-song-server --restart unless-stopped \
  -p 8099:8080 \
  -v /mnt/user/YARG/songs:/songs:ro \
  -v /mnt/cache/appdata/yarg-song-server/data:/data \
  yarg-song-server:v0.1.3-yargsong --songs /songs --data /data --listen :8080
```

Rollback is the previous image, still on the box: `registry.badassium.com/fatalexception/yarg-song-server@sha256:2d5ff48a089f...`

Verified from ENG-1 rather than from the container: a song that came from a `.yargsong` (`Thomas Arne - A-Hunting We Will Go`, playlist `Keys Packs`) is in the catalogue and `GET /song/<hash>.sng` returns 1,282,103 bytes beginning `SNGPKG` - the decoy magic does not reach a client.

The deploy took the service down for ~30 seconds, and it did not have to

The first attempt ran `docker stop` and then `docker rename`, under `set -e`. The rename failed because a `yarg-song-server-prev` from seven weeks ago still existed, so the script aborted with the service stopped and nothing started. **Stop last, or rename to a name that cannot already be taken.** The recovery used a timestamped name; the old containers are still there, which is why the collision happened at all.

Three archives in the library are reported as problems

`/api/v1/library` now carries a `problems` array naming them, which is the scanner doing its job rather than a fault:

File	Reported
`Green Day Rock Band.zip`	archive holds more than one song folder
`Rock Band 4.7z`	archive holds more than one song folder
`Rock Band 3.zip`	`zip: not a valid zip file`

The first two are the multi-song case the scanner refuses by design rather than publishing one song and dropping the rest. The third is a different thing entirely - that file is not a readable zip at all, which usually means a truncated or interrupted download.

Storage layout — where the data lives, and where it must not (2026-11-01)

This is large-file storage. The song data lives on the ARRAY with the cache enabled, and never inside the docker image. Everything the server reads or writes is a bind mount; nothing of consequence is in the container layer, and a container that stored songs in its own filesystem would lose them on every upgrade while quietly filling the docker image file.

Measured on 2026-11-01 rather than assumed:

What	Where	How it is configured
`/songs`	`/mnt/user/YARG/songs`, 25 G	`shareUseCache="yes"`, `shareCachePool="cache"`, `shareAllocator="highwater"`
`/songs/unpacked`	same share, nested `rw` mount	owner `65532:users`, mode 775
`/data`	`/mnt/cache/appdata/yarg-song-server/data`, 2.0 G	addressed as `/mnt/cache/...`, so it never leaves the cache
container layer	nothing	both are bind mounts

The library is therefore already right: writes land on the NVMe cache and the mover migrates them to the array.

`/data` is the exception, and it is deliberate rather than accidental - it is the conventional Unraid appdata placement, it avoids FUSE overhead on the pack cache's many small reads, and the mover never churns it. What it costs is that this data exists on ONE device. That is tolerable only because everything in it is regenerable: the index is rebuilt at startup and the pack cache is bounded by `pack_cache_max` and re-packs whatever it evicts. Nothing in `/data` is the only copy of anything. **If that ever stops being true, `/data` moves to the share.**

One number that did not fit, and the answer was not storage

Moving 8 GB of archives onto the share over SMB ran at **4.8 MB/s** (1,012 MB in 212 s). A share with `shareUseCache="yes"` should be absorbing writes onto NVMe at far more than that on a gigabit link, and the cache was 55% full with 1.3 T free, so it was not spilling to the array for want of room.

It was recorded unexplained, with a note to measure the write path before touching the layout, because the temptation was to conclude "the array is slow" and move the library onto the cache - a large, hard-to-reverse decision from one measurement with no diagnosis. That caution was right. Measured the same day, in this order:

Measurement	Result	Rules out
`dd` to `/mnt/user/YARG` (through shfs)	647 MB/s	the array, the share, FUSE
`dd` to `/mnt/cache`	873 MB/s	the cache pool
vault2 `eth0`	1000 Mb/s full duplex	the server's link
robocopy `/J` vs `Copy-Item`, same file, same contention	1.3 vs 1.4 MB/s	**the copy tool**

The fourth line is the one worth keeping. Buffered copying over SMB is a known bottleneck and robocopy with unbuffered I/O is the usual remedy, so that was the hypothesis - and it was wrong, measured at 1.0x. The comparison was set up as a RATIO under identical load precisely so that it could come back negative, which it did.

What was left was the client:

```
ENG-2   192.168.2.46/23 on Wi-Fi        Ethernet: Disconnected
        SSID SchillingHaus-Legacy, channel 1 (2.4 GHz), 802.11ax
        transmit rate 135 Mbps, signal 66%
ping    min 17 ms  avg 33.8 ms  max 58 ms   -- to the same subnet
```

A Wi-Fi 6E adapter parked on 2.4 GHz channel 1, with LAN round trips an order of magnitude worse than a LAN should be. 135 Mb/s is about 17 MB/s before protocol overhead, and SMB is chatty enough that 34 ms round trips account for the rest.

So the storage layout was never implicated. Nothing about the array, the cache or the docker mounts needed changing, and changing them would have made the transfer no faster while making the deployment worse. The number came from the machine doing the copying, which was not part of the system under test at all - the fourth time on this project that the instrument was the fault.